

SOLUCIÓN – Segundo Examen Parcial

Asignatura: Desarrollo de Software

SECCIÓN A – TEORÍA (2 puntos)

[1.1] ¿Cómo modelar *ShippingAddress* en un sistema de ventas según DDD?

Respuesta correcta: Como un *Value Object*, porque se identifica por sus valores y puede copiarse sin necesidad de identidad propia.

Explicación:

- La dirección de envío (calle, ciudad, país, código postal) no necesita un Id propio; su identidad se basa en la combinación de sus atributos.
- Cuando el cliente cambia su dirección en el perfil, los pedidos anteriores NO deben cambiar. Esto indica que la dirección se copia como valor en cada pedido.
- En DDD, este es el comportamiento típico de un Value Object: inmutable, definido por sus atributos y copiable sin identidad propia.

[1.2] En arquitectura hexagonal, ¿dónde colocar *ISmtpEmailSender* y *SmtpEmailSender*?

Respuesta correcta: *ISmtpEmailSender* en el núcleo (dominio o aplicación) como puerto de salida, y *SmtpEmailSender* en la capa de infraestructura como adaptador.

Explicación:

- En arquitectura hexagonal, el núcleo (dominio + aplicación) define puertos (interfaces) que expresan lo que el dominio necesita del exterior.
- *ISmtpEmailSender* es un puerto de salida: el dominio sabe que necesita enviar un correo, pero no sabe cómo se implementa.
- *SmtpEmailSender* es el adaptador concreto que implementa el puerto usando una librería SMTP; pertenece a infraestructura.
- Así se cumple Inversión de Dependencias: el núcleo depende de abstracciones y la infraestructura depende del núcleo.

SECCIÓN B – PRÁCTICA (8 puntos)

[2.1] Análisis y refactorización de BillingController

Código original:

```
public class BillingController
{
    public decimal Pay(decimal amount)
    {
        if (amount <= 0)
            return 0;

        var gateway = new StripeGateway("secret-key");
        var logger = new FileLogger("log.txt");

        var result = gateway.Process(amount);
        logger.Log("done");

        return result ? amount : 0;
    }
}
```

(1) Cinco violaciones arquitectónicas

Violación de Inversión de Dependencias (DIP)

El controlador crea directamente StripeGateway y FileLogger (clases concretas). El código de alto nivel (controlador) depende de detalles de bajo nivel (infraestructura). Si cambia el proveedor de pago (Stripe → PayPal) hay que modificar el controlador.

Acoplamiento fuerte a infraestructura (quiebra de separación de capas)

El controlador, que debería pertenecer a la capa de presentación / API, accede directamente a infraestructura (gateway de pago, logger de archivos). Debería delegar en un servicio de aplicación que use puertos (interfaces) y adaptadores.

Configuración embebida en código

La "secret-key" y el archivo "log.txt" están hardcodedos. Esto dificulta cambiar el entorno (dev/qa/prod) y va contra la idea de externalizar configuración (12-Factor App).

Lógica de negocio dentro del controlador

Reglas como "if (amount <= 0) return 0;" y "result ? amount : 0" son decisiones de negocio. Estas reglas deberían vivir en una capa de dominio o aplicación, no en la capa de presentación.

Dificultad para testear

Al crear las dependencias dentro del método no se pueden mockear fácilmente. No hay interfaces ni inyección de dependencias, por lo que las pruebas unitarias se vuelven frágiles.

Además:

- Violación de SRP (Single Responsibility Principle): el controlador valida montos, orquesta el pago y decide cómo loguear.
- Violación de Open/Closed: cambiar la forma de loguear o procesar el pago implica editar este controlador.

(2) Refactorización del diseño

Objetivo: separar responsabilidades, introducir puertos (interfaces) y adaptadores, y aplicar Inversión de Dependencias.

Interfaces necesarias

```
public interface IPaymentGateway
```

```

{
    bool Process(decimal amount);
}

public interface IAppLogger
{
    void Log(string message);
}

public interface IPaymentAppService
{
    decimal Pay(decimal amount);
}

```

Distribución por capas

- **Capa de Dominio:** contiene reglas de negocio sobre el pago (por ejemplo, validar que amount > 0).
- **Capa de Aplicación:** implementa IPaymentAppService, orquesta el flujo, depende de IPaymentGateway e IAppLogger.
- **Capa de Infraestructura:** implementa StripePaymentGateway y FileLogger, leyendo configuración externa.

Inversión de dependencias

- El controlador depende de IPaymentAppService (abstracción).
- IPaymentAppService depende de IPaymentGateway e IAppLogger (abstracciones).
- Las implementaciones concretas viven en infraestructura y se conectan mediante el contenedor de DI.

(3) Código del controlador refactorizado

```

public class PaymentAppService : IPaymentAppService
{
    private readonly IPaymentGateway _paymentGateway;
    private readonly IAppLogger _logger;

    public PaymentAppService(IPaymentGateway paymentGateway, IAppLogger logger)
    {
        _paymentGateway = paymentGateway ?? throw new ArgumentNullException(nameof(paymentGateway));
        _logger = logger ?? throw new ArgumentNullException(nameof(logger));
    }

    public decimal Pay(decimal amount)
    {
        if (amount <= 0)
        {
            _logger.Log("Invalid amount.");
            return 0;
        }

        var result = _paymentGateway.Process(amount);
        _logger.Log("Payment processed.");

        return result ? amount : 0;
    }
}

[ApiController]
[Route("api/[controller]")]
public class BillingController : ControllerBase
{
    private readonly IPaymentAppService _paymentAppService;

```

```
public BillingController(IPaymentAppService paymentAppService)
{
    _paymentAppService = paymentAppService;
}

[HttpPost("pay")]
public ActionResult<decimal> Pay([FromBody] decimal amount)
{
    var result = _paymentAppService.Pay(amount);

    if (result == 0)
        return BadRequest("Payment failed or invalid amount.");

    return Ok(result);
}
```

El controlador ahora delega en un servicio de aplicación, no crea dependencias concretas y no contiene lógica de negocio compleja.

[2.2] Tests para TransactionDomainService y AuthorizationAppService

Resumen del código relevante: Value Object Amount, agregado Account, IAccountRepository, IExternalNotificationService, TransactionDomainService y AuthorizationAppService.

(1) Tests para TransactionDomainService

```
public class TransactionDomainServiceTests
{
    [Fact]
    public async Task ExecuteWithdrawalAsync_ExistingAccount_WithdrawsAndSaves()
    {
        // Arrange
        var account = new Account(id: "ACC-123", balance: 100m, isActive: true);

        var repoMock = new Mock<IAccountRepository>();
        repoMock
            .Setup(r => r.FindByIdAsync("ACC-123"))
            .ReturnsAsync(account);

        var service = new TransactionDomainService(repoMock.Object);

        // Act
        await service.ExecuteWithdrawalAsync("ACC-123", 40m);

        // Assert
        Assert.Equal(60m, account.GetBalance());
        repoMock.Verify(r => r.SaveAsync(account), Times.Once);
    }

    [Fact]
    public async Task ExecuteWithdrawalAsync_AccountNotFound_ThrowsInvalidOperationException()
    {
        // Arrange
        var repoMock = new Mock<IAccountRepository>();
        repoMock
            .Setup(r => r.FindByIdAsync("ACC-404"))
            .ReturnsAsync((Account?)null);

        var service = new TransactionDomainService(repoMock.Object);

        // Act & Assert
        await Assert.ThrowsAsync<InvalidOperationException>(
            () => service.ExecuteWithdrawalAsync("ACC-404", 10m));

        repoMock.Verify(r => r.SaveAsync(It.IsAny<Account>()), Times.Never);
    }
}
```

Estos tests verifican la regla de negocio y el uso del repositorio, siguiendo el patrón Arrange–Act–Assert y mockeando solo las dependencias.

(2) Tests para AuthorizationAppService

```
public class AuthorizationAppServiceTests
{
    [Fact]
    public async Task AuthorizeAndNotifyAsync_WhenNotificationSucceeds_ReturnsTrue()
    {
        // Arrange
        var domainServiceMock = new Mock<TransactionDomainService>(MockBehavior.Strict, (IAccountRepository) null);
        domainServiceMock
            .Setup(s => s.ExecuteWithdrawalAsync("ACC-1", 50m))
            .Returns(Task.CompletedTask);

        var notificationMock = new Mock<IExternalNotificationService>();
```

```

notificationMock
    .Setup(n => n.SendNotificationAsync("ACC-1", It.IsAny<string>()))
    .ReturnsAsync(true);

var appService = new AuthorizationAppService(domainServiceMock.Object, notificationMock.Object);

// Act
var result = await appService.AuthorizeAndNotifyAsync("ACC-1", 50m);

// Assert
Assert.True(result);
domainServiceMock.Verify(s => s.ExecuteWithdrawalAsync("ACC-1", 50m), Times.Once);
notificationMock.Verify(n => n.SendNotificationAsync("ACC-1", It.IsAny<string>()), Times.Once);
}

[Fact]
public async Task AuthorizeAndNotifyAsync_WhenNotificationFails_ReturnsFalse()
{
    // Arrange
    var domainServiceMock = new Mock<TransactionDomainService>(MockBehavior.Strict, (IAccountDomainServiceMock)
        .Setup(s => s.ExecuteWithdrawalAsync("ACC-2", 20m))
        .Returns(Task.CompletedTask);

    var notificationMock = new Mock<IExternalNotificationService>();
    notificationMock
        .Setup(n => n.SendNotificationAsync("ACC-2", It.IsAny<string>()))
        .ReturnsAsync(false);

    var appService = new AuthorizationAppService(domainServiceMock.Object, notificationMock.Object);

    // Act
    var result = await appService.AuthorizeAndNotifyAsync("ACC-2", 20m);

    // Assert
    Assert.False(result);
    domainServiceMock.Verify(s => s.ExecuteWithdrawalAsync("ACC-2", 20m), Times.Once);
    notificationMock.Verify(n => n.SendNotificationAsync("ACC-2", It.IsAny<string>()), Times.Once);
}

[Fact]
public async Task AuthorizeAndNotifyAsync_WhenDomainThrows_ExceptionIsPropagated()
{
    // Arrange
    var domainServiceMock = new Mock<TransactionDomainService>(MockBehavior.Strict, (IAccountDomainServiceMock)
        .Setup(s => s.ExecuteWithdrawalAsync("ACC-3", 200m))
        .ThrowsAsync(new InvalidOperationException("Insufficient funds."));

    var notificationMock = new Mock<IExternalNotificationService>();

    var appService = new AuthorizationAppService(domainServiceMock.Object, notificationMock.Object);

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(
        () => appService.AuthorizeAndNotifyAsync("ACC-3", 200m));

    notificationMock.Verify(n => n.SendNotificationAsync(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
}
}

```

Los tests de la capa de aplicación verifican la coordinación entre dominio y servicio externo, sin re-testear la lógica interna del agregado.

SECCIÓN C – MODELO TÁCTICO DDD (10 puntos)

Contexto: PharmaLogistics S.A. debe gestionar envíos de medicamentos en contenedores refrigerados, control de temperatura, lecturas de sensores y validaciones reglamentarias externas.

(1) Value Objects e invariantes

Se identifican tres Value Objects principales:

- **TemperatureRange** (MinCelsius, MaxCelsius), con invariante $\text{MinCelsius} \leq \text{MaxCelsius}$.
- **Volume** (Value), con invariante $\text{Value} > 0$ para representar volúmenes de medicamentos y capacidades de contenedores.
- **SensorReading** (Timestamp, Temperature), para representar una lectura inmutable de sensor asociada a un contenedor.

(2) Agregados, entidades e invariantes

Agregado Shipment

- Aggregate Root: Shipment, con Id, Status (PENDIENTE, EN_TRANSITO, COMPROMETIDO, etc.), colección de Medicine, RangoTemperaturaRequerido, TotalVolume y ContenedorAsignadoId.
- Entidad interna Medicine, con nombre, Volume y TemperatureRangeRequerido.
- Invariantes: el rango de temperatura requerido es la intersección de los rangos de todos los medicamentos; el volumen total es la suma de los volúmenes; no se puede pasar de PENDIENTE a EN_TRANSITO sin contenedor asignado y validado.

Agregado Container

- Aggregate Root: Container, con Id, Status (DISPONIBLE, EN_TRANSITO, COMPROMETIDO, etc.), VolumenMaximo, TemperatureRangeOperativo, ShipmentAsignadoId y opcionalmente lecturas de SensorReading.
- Responsabilidades: registrar lecturas, marcarse COMPROMETIDO si una lectura sale del rango, y asignarse a un envío mediante un método AssignShipment que valida capacidad y rango de temperatura.
- Invariantes: si el contenedor está EN_TRANSITO debe tener un ShipmentAsignadoId; si está COMPROMETIDO no puede pasar directamente a DISPONIBLE sin un proceso explícito; cualquier lectura fuera de rango implica estado COMPROMETIDO.

(3) Servicios de dominio

- **AssignContainerToShipmentService**: coordina el Proceso 1, buscando un contenedor DISPONIBLE que cumpla con capacidad y rango de temperatura para un envío PENDIENTE, y asignándolo al Shipment.
- **ContainerMonitoringService**: implementa los Procesos 2 y 3, registrando lecturas de sensor, marcando contenedores como COMPROMETIDOS y actualizando el estado de los Shipments afectados (idealmente mediante eventos de dominio).
- **ShipmentDepartureValidationService**: implementa el Proceso 4, validando que el contenedor asignado sigue apto y consultando IRegulacionesRepository para comprobar que la ruta es legal antes de cambiar el estado del envío a EN_TRANSITO.

Este modelo táctico DDD mantiene las invariantes en los agregados Shipment y Container, reutiliza Value Objects con reglas claras (TemperatureRange, Volume, SensorReading) y delega en servicios de dominio la coordinación entre agregados y con sistemas externos.